ProEm: Protocol Emulator ASIC

Design Reference

Paul Vincent Bezzina

17 October 2026

Process	IHP SG13CMOS5L 130 nm, through Tiny Tapeout (6 × 4 tiles)
System clock	50 MHz (10BASE-T transmit needs 40 or 60 MHz)
Channels	2 independent protocol channels, A and B
Host link	UART in (auto-baud), UART out (rate set by `LINK`)
Standards verified	UART, SPI (controller in all four modes, and target), I^2C (controller and target), PS/2, 10BASE-T receive, Manchester transmit, USB low speed receive and transmit with CRC and handshake, SWD-style acknowledge handling, CAN with bit stuffing, arbitration, CRC-15 and acknowledge, and 16 more in the *Standards Programming Guide* (RS-485 to RC5 infrared)
Logic	18,969 cells after synthesis, 25,612 after place and route (43 % of the core)
Timing	met at all three corners after routing; 3.84 ns setup slack at 1.08 V, 125 °C (62 MHz)
Instruction set	ISA v0.7.1 (separate document: *ProEm Instruction Set*)

Contents

1 Overview **3**

2 Top-level interface **5**

3 Blocks **5**

3.1 UART receiver and auto-baud (`uart_rx`) . . . 5

3.2 Control unit (`control_unit`) . . . 5

3.3 Frame chain (`frame_chain`) . . . 6

3.4 Host FIFO and entry dispatcher . . . 7

3.5 Line codec (`line_codec`) . . . 7

3.6 Return path (`return_path`, `uart_tx`) . . . 8

4 Host protocol **8**

4.1 Start-up . . . 8

4.2 Return messages . . . 9

4.3 Choosing `RUN` modes . . . 9

5 Protocol recipes **9**

6 Verification **11**

7 Implementation results **14**

7.1 Synthesis 14

7.2 Place and route on Tiny Tapeout 14

7.3 Pin map 15

8 Limits and future work **15**

9 Files **16**

1 Overview

ProEm is an open-source protocol emulator ASIC. The chip lets an RP2040 speak many serial protocols through one piece of hardware. The RP2040 does not bit-bang anything. It describes a frame to the chip in a few instruction bytes: how the frame starts, which fields follow, which way each field goes, how the frame ends, how the line is coded and where the bit clock comes from. The chip then runs the frame at up to one bit per system clock, with exact timing, and sends back what it received together with a status for every frame.

The design has four parts:

1. **Host side.** A UART receiver with auto-baud feeds the control unit, which decodes ISA v0.7.1 instructions into registered configuration writes. A return path sends received data, frame status and link events back on a second UART.
2. **Input pipeline.** Every input pin passes through two synchroniser flip-flops and one more register before any logic uses it.
3. **Two channels.** Each channel has a host FIFO for write data, a *frame chain* that runs the frame, an *entry dispatcher*, and a *line codec* that turns pins into bits and ticks, and bits into pin levels.
4. **Outputs.** Each channel drives a data output, a pair output (the complement, for USB D−), a clock output and a select output, each with an output enable. Bidirectional pads are read back through the input pins.

The two channels run at the same time. That is how the chip does SPI full duplex (one channel writes MOSI, the other reads MISO), UART receive and transmit together, or a transmitter looped into a receiver for self-test.

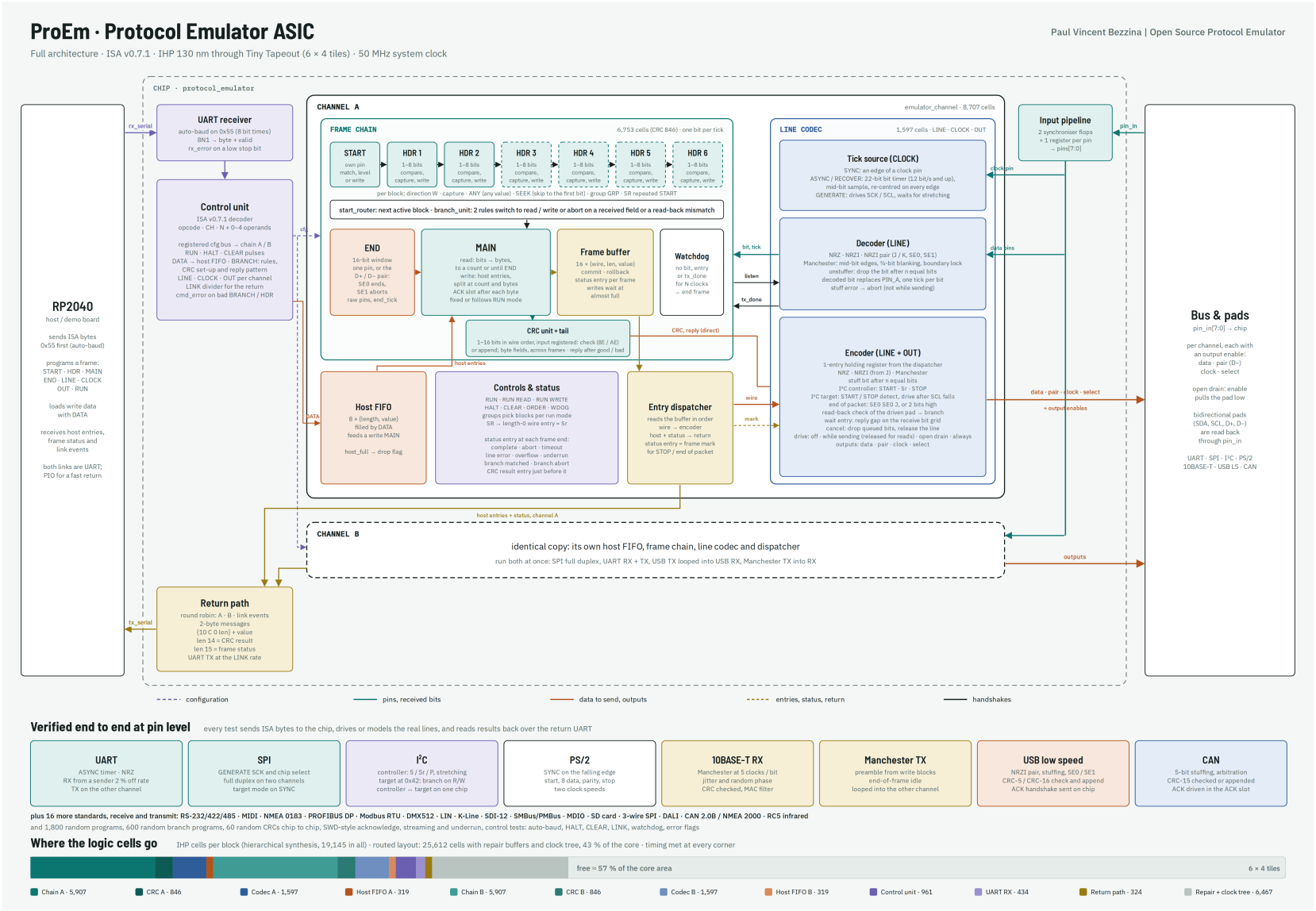


Figure 1: The whole chip. Channel A is shown in full; channel B is an identical copy.

2 Top-level interface

The top module is `protocol_emulator`. Its ports are generic; the Tiny Tapeout wrapper maps them onto `ui_in`, `uo_out` and the bidirectional `uio` pins.

Port	Dir	Meaning
`clk`, `rst_n`	in	System clock and active-low asynchronous reset
`rx_serial`	in	Instruction bytes from the RP2040, 8N1. The first byte after reset must be `0x55`
`pin_in[7:0]`	in	Protocol input pins, shared by both channels
`tx_serial`	out	Return UART to the RP2040
`data_out_x`, `data_oe_x`	out	Channel x data output and output enable. In open-drain mode `out` is 0 and `oe` pulls the pad low
`pair_out_x`, `pair_oe_x`	out	Complement of the data output (USB D−)
`clock_out_x`, `clock_oe_x`	out	Generated clock (SCK, SCL)
`select_out_x`	out	Chip select: active from the frame's START until its last bit has gone out
`armed_x`, `host_full_x`, `baud_locked`	out	Debug status

Pads. A bidirectional line such as SDA is a pad driven by `data_out`/`data_oe` and read back on one of the `pin_in` bits. I^2C uses open drain on both SDA and SCL, so the pads need external pull-ups. USB low speed needs 1.5 kΩ from D− to 3.3 V when the chip is the device, and the pad voltage must reach 2.8 V or a transceiver is needed.

Input latency. The pipeline adds three clocks between a pad and the logic. Every pin is delayed equally, so a data pin and a clock pin stay aligned.

3 Blocks

3.1 UART receiver and auto-baud (`uart_rx`)

After reset the receiver waits for the first falling edge, then counts clocks until the fifth falling edge. For `0x55` those five edges are exactly 8 bit times apart, so the clocks per bit are the count divided by 8. The receiver then waits for the stop bit and becomes a normal 8N1 receiver at that rate. The sync byte is not passed on. A byte whose stop bit is low raises `rx_error`. The measured rate is also the default rate of the return UART.

3.2 Control unit (`control_unit`)

The control unit reads an opcode byte {`OP, CH, N`}, collects 0–4 operand bytes, and in one further clock turns the instruction into:

- a write on the shared configuration bus with a write enable for channel A or B (`START`, `HDR`, `MAIN`, `END`, `WDOG`, `ORDER`);
- a pulse: `RUN` with its mode, `HALT` or `CLEAR`;
- a host FIFO write (`DATA`);
- a register update: `LINE`, `CLOCK` and `OUT` per channel, `LINK` for the return link.

Every output is a register, so the configuration bus forms a pipeline stage and the decode logic is never in a channel's timing path. `START` patterns are given as plain numbers; the control unit aligns

them for the channel's bit order. `BRANCH` rules travel on the same bus (select 8 and 9). `BRANCH` with `N` > 4 and `HDR 0`/`HDR 7` raise `cmd_error`.

3.3 Frame chain (`frame_chain`)

The frame chain is the frame engine: one START block, up to six header blocks and one MAIN block, run strictly in order, one bit per tick.

- **START** slides an 8-bit window over its own pin and matches a pattern of 1–8 bits; with 0 bits it waits for a level (SPI chip select); as a write block it sends its value when `RUN` starts the frame.
- **Headers** each compare, capture or send a 1–8 bit field. Each has a group (always, read runs, write runs) and an optional repeated START before it. A read header with `ANY` accepts any value and keeps what arrived, in place of its compare value.
- **start_router** enables the next active block after the previous one succeeds. It skips unconfigured blocks and blocks whose group is not in the current run mode.
- **branch_unit** holds two `BRANCH` rules. When a watched block succeeds, a rule compares its value under a mask and chooses an action: keep the mode, switch to read, switch to write, or abort. A second kind of rule fires when the codec reports that the line read back differently from what the chip drove. The decision is registered, and the chain pauses for that one clock so the next block starts in the new mode. A branch abort keeps the values captured so far and stops the codec driving at once.
- **MAIN** reads bits into bytes, to a count or until END; or writes host entries, splitting them at the frame's bit count and, with the ACK slot on, at byte edges. After each byte it can add one bit the other way (I^2C ACK). Its direction is fixed or follows the run mode.
- **END** slides a 16-bit window over one raw pin, or over the D+/D− pair compare: both low (SE0) is a 1, both high (SE1) aborts with a line error. After an abort, START waits until END has matched again, so it never searches inside the rest of a rejected frame.
- **crc_unit** holds one CRC register of 1 to 16 bits, kept top aligned in 16 bits so every width uses the same shifter. It takes up to 8 bits per clock in wire order: single received bits as they are sampled (after unstuffing), the START value when it matches, and each written entry as it enters the frame buffer. The chain registers what it feeds, so the register takes the bits one clock after they are sampled or written. It is reloaded with its initial value whenever no frame is running. Its `good` output compares the register with the residue; its `crc_out` output is the register, inverted if asked.
- **Tail** runs after MAIN ends, before the frame completes, when the CRC or the reply is configured. Each step waits until the CRC register has taken every earlier bit, so the tail starts one clock after MAIN's last bit. It has up to six steps, each done once: the CRC result entry (read frames with check on), two CRC entries of up to 8 bits (write frames with append on), the reply gap, and two reply entries. The result goes into the frame buffer as a host entry; the CRC, gap and reply entries go straight to the codec once every earlier wire entry of the frame has been handed over, so they are never queued behind received data waiting for the return link. The reply is decided at the gap step, after the last received bit has reached the CRC. A rule with `SRC` $= 1$, `BLOCK` $= 1$ fires when the result entry is pushed.
- **Watchdog** ends a frame when no bit, host entry or sent entry is seen for a set number of clocks.
- **Frame buffer** holds 16 entries {`wire, length, value`}. Written values are released at once; captured values are released when MAIN starts and dropped on abort. At the end of every

frame the chain adds a *status entry* (length 15) with complete, abort, timeout, line error, overflow, underrun, branch-matched and branch-abort flags.

Read blocks only take ticks once every bit written ahead of them has left the chip (`line_free`), so a frame can send a request and then read the reply. Write blocks wait when the buffer is almost full, so written bits are never lost; received entries that arrive with the buffer full are dropped and the frame's status has the overflow flag.

Streaming and underrun. A write MAIN takes host entries as they arrive, so `DATA` can keep coming during the frame. With a generated clock the line simply pauses when the host FIFO is empty. On a line with fixed timing (ASYNC, RECOVER, SYNC) the chain aborts with the *underrun* flag when MAIN needs data, none is waiting and every queued bit has gone out, and the codec stops driving. While a read-back `BRANCH` rule is set, a write frame also stays open until its last bit has left the pin, so an arbitration loss in the last bits still reaches the frame's status.

3.4 Host FIFO and entry dispatcher

The host FIFO holds 8 {`length, value`} entries loaded by `DATA` and feeds a write MAIN. A `DATA` arriving while it is full is dropped and reported as a link event.

The entry dispatcher reads the frame buffer in order. Wire entries go to the codec's encoder; host entries and status entries go to the return path. After a cancel (branch abort, underrun, I^2C bus error) it discards the frame's remaining wire entries up to its status entry, and the chain's count of bits in flight is kept consistent. When a status entry reaches the head, every bit of that frame has already been handed to the encoder, so the dispatcher gives the encoder a *frame mark* at that moment. The encoder uses it to send the I^2C STOP or the end of packet at exactly the right place.

3.5 Line codec (`line_codec`)

The codec sits between the pins and the frame chain. `CLOCK` chooses the tick source, `LINE` the line code, `OUT` the output behaviour. The separate *Line Codec* reference (`docs/line_codec.pdf`) explains it signal by signal.

Tick source.

- **SYNC**: a tick on the rising, falling or both edges of a clock pin (SPI target, PS/2, a read-back clock).
- **ASYNC / RECOVER**: a bit timer of `DIV` clocks per bit that samples mid-bit and re-centres on every edge of the data line. The decoded bit replaces `LINE`'s `PIN_A` in the pins the chain sees, with one tick per bit.
- **GENERATE**: the codec drives the clock output, `DIV` clocks per half period, only while a bit is waiting to go out or a read block wants one. The sampling edge gives the tick. With open drain it waits for the clock read-back pin to go high, which gives I^2C clock stretching.

Decoder. NRZ takes the mid-bit sample. NRZI compares it with the previous one (same = 1). The NRZI pair code reads J/K from D+ and D− and gives no bits during SE0 or SE1. Manchester decodes on the mid-bit edges: the new level is the bit, and edges within $\frac{3}{4}$ of a bit after an accepted edge are ignored, which skips the boundary edges between equal bits. An unstuffer drops the bit after n equal bits; a stuffed bit with the wrong value aborts the frame with a line error.

Encoder. It holds one entry from the dispatcher and sends it bit 0 first (the chain has already reversed MSB-first fields). It encodes NRZ, NRZI starting from J, or Manchester, inserts a stuff bit after n equal bits, and continues seamlessly into the next entry. In I^{2}C mode it sends START before a transaction, repeated START for a length-0 entry, and STOP at the frame mark. With end of packet on, the frame mark produces two bit times of SE0 and one of J on a pair, or two bit times high on a single line. `tx_done` after each entry lets the chain know the line is free.

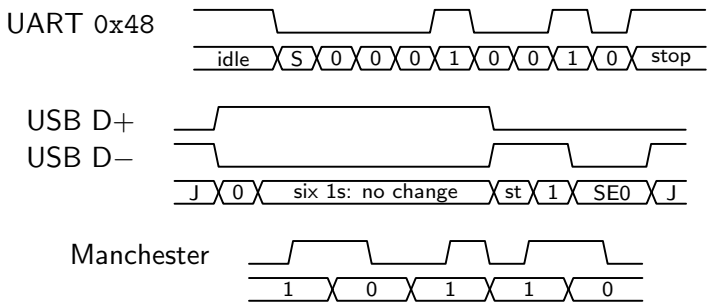


Figure 2: Three line codes from the same codec. UART: NRZ, LSB first, start and stop bits. USB low speed: NRZI on D+/D− (a 0 changes the line), a stuff bit (st) after six 1s, then the end of packet. 10BASE-T: Manchester, low-to-high in the middle of the bit = 1.

I^{2}C target. With `OUT I2C` and a SYNC clock on SCL, the codec detects START and STOP (SDA falling or rising while SCL is high), which start the frame and end a MAIN that waits for END. A bit to send waits for SCL to fall before it is driven, and SDA is released after the falling edge that ends it, so the address ACK and data bytes meet the I^{2}C set-up and hold rules. The chip never drives SCL in this mode.

Read-back check and cancel. While sending, the codec compares the pad it drives (read back on `LINE`'s `PIN_A`) with the level it wants, in the middle of each bit. A difference is passed to the branch unit (CAN-style arbitration). On a cancel the encoder drops its current and next entry, releases the line and counts the dropped entries as sent.

Outputs. `DRIVE` is off after reset, push-pull while sending, open drain, or push-pull always (with an idle level, for a UART line). The pair output carries the complement. The select output is active from the frame's START until its last bit has gone out.

3.6 Return path (`return_path`, `uart_tx`)

The return path takes host entries and status entries from both channels and link events, in round robin, and sends each as a two-byte message on the return UART at the `LINK` rate (or the auto-baud rate when `LINK` is 0).

4 Host protocol

4.1 Start-up

1. Release reset and hold `rx_serial` high for at least one byte time.
2. Send `0x55`. The chip measures the rate from it; `baud_locked` goes high.
3. Optionally send `LINK` to speed up the return link (`E0 00 04` = 4 clocks per bit, 12.5 Mbaud at 50 MHz).

4. Program each channel: `LINE`, `CLOCK`, `OUT`, `ORDER`, `WDOG`, the blocks, then `RUN`.

`CLEAR` returns a channel to its reset state, including its outputs; `HALT` stops a frame and disarms the channel but keeps its program.

4.2 Return messages

First byte	Second	Meaning
`10 C 0 LLLL`	value	Host entry from channel `C`, length 0–8. Length 0 with value `02` is a NACK from a writing MAIN
`10 C 0 1110`	result	CRC result of a read frame: bit 0 good, bit 1 checked. Comes after the frame's data and before its status
`10 C 0 1111`	flags	End of a frame on channel `C`: bit 0 complete, 1 abort, 2 timeout, 3 line error, 4 overflow, 5 underrun, 6 branch matched, 7 branch abort
`11 000000`	flags	Link events: bit 0 `rx_error`, 1 `cmd_error`, 2 `DATA` dropped on A, 3 on B

A frame's host entries always come before its status message. A simple receiver on the RP2040:

```
void on_return_message(uint8_t head, uint8_t value) {
    if ((head & 0xC0) == 0xC0) { link_events(value); return; }
    int chan = (head >> 5) & 1, len = head & 0x0F;
    if (len == 15) frame_done(chan, value);        /* status flags   */
    else if (len == 14) crc_result(chan, value);   /* bit 0 good     */
    else           frame_data(chan, len, value);   /* 0..8 bits      */
}
```

4.3 Choosing `RUN` modes

- **Receiving:** plain `RUN` arms the channel; it stays armed after every frame.
- **Packets** (USB, Ethernet, I²C transactions): load the data with `DATA`, then `RUN WRITE` or `RUN READ`. The channel starts exactly one frame per `RUN`.
- **Streams** (UART transmit): plain `RUN`. The channel also starts a new frame whenever data is waiting.
- **Long packets**: load a few `DATA` entries, send `RUN WRITE`, then keep sending `DATA` while the frame runs. Watch `HOST_FULL`; on a fixed-timing line keep ahead of the line or the frame ends with an underrun (status `22`).
- **Reacting to the other side** (I²C target, SWD, arbitration): add `BRANCH` rules; the status byte's bits 6 and 7 say which way the frame went.

5 Protocol recipes

Each recipe lists the instruction bytes. The bytes were produced by the test bench's own encoder, and the same programs (with smaller dividers, to keep simulation short) pass the tests in Section 6. Dividers are for a 50 MHz clock.

UART receive, 115 200 baud, channel A. RX on pin 0. `LINE` none on pin 0; `CLOCK` ASYNC, 434 clocks per bit; START reads the start bit; MAIN reads 8 data bits and the stop bit.

```
A0 00 00   B0 40 00 01 B2   10 01 00 00   30 00 00 09   70
```

UART transmit, channel B. Push-pull output, idle high; START writes the start bit; MAIN writes 9 bits. Each byte is two `DATA` entries: the byte and a stop bit.

```
A8 00 00   B8 40 00 01 B2   D8 E0   18 81 00 00   38 40 00 09
CF 48   C8 01   78                                  ("H", then RUN)
```

SPI controller, mode 0, MSB first, full duplex. Channel A generates SCK (half period 6 clocks) and chip select, writes MOSI from `DATA`. Channel B reads MISO on pin 3, clocked by SCK read back on pin 1, started by chip select read back on pin 2.

```
A: 61   A0 00 00   B0 81 00 00 06   D0 C0   50 0F A0   10 80 00 00   30 40 00 10
B: 69   A8 03 00   B8 01 00 00 00   18 00 02 00   38 03 00 10   78
transfer:   C7 A5   C7 5A   70
```

I²C controller, 100 kHz, register read and write in one program. Device 0x48, register 0x05. SDA read back on pin 0, SCL on pin 1; open drain with I²C conditions.

```
61   A0 00 00   B0 89 00 00 FA   D0 A4   50 EA 60
10 88 00 90   21 01 00 00   22 88 00 05   23 01 00 00
24 98 80 91   25 11 00 00   30 90 00 10
write:  C7 A1   C7 B2   72        bus: S 90 A 05 A A1 A B2 A P
read:   71                        bus: S 90 A 05 A Sr 91 A d0 A d1 N P
```

PS/2 keyboard to host, channel B. Clock on pin 4 (falling edge), data on pin 5. Each key gives {`8, code`} and {`2, parity + stop`}.

```
A8 05 00   B8 14 00 00 00   18 01 05 00   38 05 00 0A   78
```

10BASE-T receive, broadcast frames. Manchester line on pin 0; END sees the line high for 8 clocks at the end of the frame; return link at 4 clocks per bit.

```
E0 00 04   A0 60 00   B0 C0 40 00 05   10 08 00 D5
21 48 00 FF   22 48 00 FF   23 48 00 FF   24 48 00 FF   25 48 00 FF   26 48 00 FF
30 20 00 00   40 40 00 00 FF   70
```

Manchester transmit (6 clocks per bit, a 60 MHz clock for 10 Mb/s). The preamble comes from START and six write headers; MAIN sends up to 8 `DATA` bytes (SFD first); end of packet holds the line high for two bit times.

```
A0 60 00   B0 C0 00 00 06   D0 48   10 88 00 55
21 88 00 55   22 88 00 55   23 88 00 55   24 88 00 55   25 88 00 55   26 88 00 55
30 40 00 40      then DATA entries and RUN WRITE (72)
```

USB low speed receive. D+ on pin 0, D− on pin 1; NRZI pair with unstuffing; END on the pair, 2 bit times of SE0, SE1 aborts.

```
E0 00 04   A0 40 31   B0 C0 80 00 21   10 08 00 80   30 20 00 00
40 10 A1 00 03   70
```

USB low speed transmit, channel B. SYNC from START, PID `0x4B` from a header, 32 data bits from `DATA`, stuffing and end of packet.

```
A8 40 31   B8 C0 00 00 21   D8 58   18 88 00 80   29 88 00 4B   38 40 00 20
CF FF   CF FF   CF 7E   CF 00   7A
```

I²C target at address 0x42. SDA on pin 0, SCL on pin 1. The R/W bit is captured with `ANY`; the branch makes MAIN write (the controller reads) or read (the controller writes); MAIN ends at STOP or repeated START.

```
61   A0 00 00   B0 01 00 00 00   D0 84   10 00 00 00   21 07 00 42
22 41 40 00   23 81 00 00   F0 92 01 01   30 B0 00 40   70
data for reads:   C7 5A   C7 C3
```

SWD-style read with acknowledge check. SWCLK generated (1 MHz), SWDIO open drain on pin 0, LSB first. Only an OK acknowledge continues to the 33-bit data phase; WAIT and FAULT abort with the acknowledge returned.

```
60   A0 00 00   B0 81 00 00 19   D0 80   10 88 00 A5
21 01 40 00   22 43 40 00   F0 72 07 01   30 00 00 21   70 (each transaction)
```

CAN-style arbitration. 1 Mbit/s (50 clocks per bit), open drain, read back on pin 0. A lost arbitration aborts the frame and releases the line in the same bit (status `C2`).

```
61   A0 00 00   B0 40 00 00 32   D0 80   10 81 00 00   30 40 00 13   F0 C8 00 00
C7 id[10:3]   C2 id[2:0]   C7 data   72
```

6 Verification

The test bench (`sim/bench.h`) simulates the whole chip with Yosys CXXRTL. Every test does what the RP2040 would do: it resets the chip, sends `0x55`, sends instruction bytes on `rx_serial`, and decodes the return UART on `tx_serial` into host entries, frame status and link events. Protocol lines are driven or modelled at the pins every clock: open-drain buses are wired-AND, clocks are looped back through `pin_in`, and devices are behavioural models.

Test	What it checks	Result
`t_control`	Auto-baud at 16 and 7 clocks per bit; read frames; header mismatch; watchdog timeout; `HALT`, re-`RUN`; `CLEAR`; START alignment in both bit orders; `LINK`; `cmd_error`, `rx_error`; `DATA` dropped on a full FIFO	Pass
`t_uart`	Channel A receives 5 bytes from a sender 2 % fast, then 2 % slow; channel B transmits “Hi!” decoded by a model	Pass
`t_spi`	Controller full duplex against a slave model; target mode against a master model; SCK half period 4, 6, 12 clocks	Pass
`t_i2c`	Device model on an open-drain bus: `RUN WRITE`, `RUN READ` with Sr, clock stretching, address NACK with STOP; SCL half period 4, 10, 25	Pass
`t_ps2`	Four keys with parity and stop bits at two clock rates	Pass
`t_ethernet`	Manchester line timed in nanoseconds with ±3 ns jitter and random phase at 50 and 60 MHz; all broadcast frames intact and CRC checked, other addresses rejected; a 64-byte frame reported as overflow	Pass
`t_manchester`	Manchester transmit at 6 and 10 clocks per bit, decoded by the other channel; end-of-frame idle and release	Pass
`t_usb`	Receive with stuffing at 33.33 clocks per bit; SE1 and missing stuff bit abort with line error; transmit decoded by a model, end of packet SE0 SE0 J then released; channel B transmit looped into channel A receive	Pass
`t_random`	1,800 random programs (6 seeds × 150, both bit orders): 0–6 headers, fields of 1–8 bits, capture on and off, random data, mismatches, splits across frames	Pass

Test	What it checks	Result
`t_i2c_target`	Chip as I^2C target at 0x42 against a controller model: write of two bytes, read of two bytes (branch to write, controller NACK), wrong address (no ACK, abort), register write then repeated START and read; SCL at 417 kHz and 1 MHz	Pass
`t_branch`	SWD-style acknowledge: OK continues to 33 data bits, WAIT and FAULT abort with the acknowledge returned. CAN-style arbitration on an open-drain line against a node model: the chip wins, the chip loses and releases the line within the bit, and the channel is reusable afterwards	Pass
`t_stream`	Underrun: 8 of 32 bits loaded on a UART line aborts with status `22`. UART stream of 12 bytes (36 entries) fed while the frame runs. SPI stream of 20 bytes with the generated clock pausing	Pass
`t_standards`	Sixteen further standards, receive and transmit, each against a model of the other side (42 checks): RS-232/422/485 (driver enable on the select output), MIDI, NMEA 0183, PROFIBUS DP, Modbus RTU (CRC checked on chip with `BYTE` and `SPAN`), DMX512 (break detection by a framing abort), LIN master and slave (seeking header across a gap), K-Line fast and 5-baud init, SDI-12, SMBus with PEC (checked on chip), MDIO (PHY switching 0 ns after MDC, push-pull), SD card CMD0 and CMD8 with the CRC-7 from the chip, 3-wire SPI (push-pull, no overlap), DALI (Manchester decoder with boundary lock), CAN 2.0B extended frames (NMEA 2000) and RC5 infrared (1.778 ms bits, 22-bit divider). The programs are listed in the *Standards Programming Guide*	Pass
`t_crc`	USB DATA0 receive with CRC-16: good packet answered with an ACK handshake 3 bit times after the end of packet, corrupted packet gets no reply, next packet good again; USB token CRC-5 good and bad; USB DATA1 transmit with CRC-16 appended and inverted, decoded by a model; CAN frame with bit stuffing from a node model, CRC-15 good (dominant ACK in the ACK slot), corrupted (no ACK), and a heavily stuffed ID; CAN chip to chip on a wired-AND bus, B appends the CRC and A checks it and acknowledges; 60 random CRCs (width 1–16, random polynomial, init, inversion, bit order and length, a third corrupted on the wire) looped from B to A	Pass
`t_random_branch`	600 random branch programs (3 seeds × 100, both bit orders): a capture-any header of 1–8 bits, random mask and value (half forced to match), random hit and miss actions, random base run mode, read-group and write-group headers; checks the path taken, the captures and the status byte	Pass
`t_modes`	SPI controller in all four modes at 3 and 6 clocks per half period, writing into a device model that samples with no hold margin and reading its reply; both `BRANCH` rules aborting on their own value; `NOP` accepted without an error	Pass

Tiny Tapeout test. `test/test.py` (cocotb, Icarus Verilog) drives the Tiny Tapeout top level through its pads: auto-baud and the `cmd_error` event; a UART loopback from channel B's data pad into channel A; an SPI loopback in which channel A generates SCK and MOSI on its pads and channel B samples them back, gated by channel A's chip select; an I^2C loopback in which channel A is the controller and channel B the target at 0x42 on the same open-drain wire; sending "A0000" over UART with the host library; and a CRC-8 loopback, channel B appending the CRC and channel A checking it, with the right residue (result `03`) and a wrong one (`02`). It passes on the RTL and is run on the gate-level netlist by the `gds` workflow.

Bugs found by these tests and fixed.

- The frame buffer's full check compared a 32-bit difference; after the pointers wrapped it never saw "full" and overwrote unread entries.
- Header and START pushes are registered, so write backpressure must stop at "almost full".
- Chip select glitched between a write frame completing and its last bits reaching the codec.
- STOP and the end of packet waited for the return link; they now follow the frame mark.
- Outputs drove low after reset; they are now off until `OUT` is given.
- A write frame finishes as soon as its last bits are queued, before they leave the pin, so a read-back rule had no frame left to act on (found by the CAN-style test). Write frames now stay open until the line drains while such a rule is set.
- The CAN start-of-frame bit was dropped as a stuff bit when it followed five or more idle recessive bits; unstuffing now counts only inside a frame (found by the CAN CRC test).
- A transmit with the chain's own data pin not read back raised a stuff error and aborted the frame; stuff errors are now ignored while the chain itself is driving the line.
- The reply was queued in the frame buffer behind received data and reached the line about 10 bits late on CAN; the tail now hands its wire entries straight to the codec.
- Timing: the first branch design fed the new mode straight into MAIN in the same clock, adding about 5 ns to the worst path. The decision is now registered and the chain pauses for one clock.
- A second cancel within two clocks of the first, or an entry finishing on the same clock as a pending drop, lost one `tx_done`; the channel then never listened again until `CLEAR` (found by the line codec proof, v0.7.1).
- In SPI modes 1 and 3 the generated-clock path changed the data pin on the device's sampling edge, leaving no hold time; the data is now held through that edge (found through a coverage gap, v0.7.1).

Further checks. The verification report (`docs/verification_report.pdf`) records the full sign-off of v0.7.1, and the toolchain guide explains how to run each check.

- **Formal proofs.** Six modules are proven with SymbiYosys for every input sequence and with no depth limit (ABC `pdr`); each harness in `formal/` wraps the unchanged RTL with a small reference model. The host FIFO and the frame buffer: occupancy and flags exact, data out unchanged and in order, rollback discarding exactly the uncommitted entries. The UART transmitter: every frame exact for every ratio from 1 to 15. The branch unit: the two rules behave as the ISA describes. The CRC unit: the 8-bit-per-clock update equals a bit-by-bit reference CRC for every width, polynomial and chunk length. The line codec: every accepted entry is answered with exactly one `tx_done`, whatever the cancels, and the output enables obey `DRIVE`.
- **Lint.** Verilator with every warning on (`scripts/lint.sh`) is clean; 13 reviewed waivers are recorded in `lint/waivers.vlt`. There are no combinational loops (Yosys `check -assert`).
- **Second simulator and coverage.** Every pin-level test also runs on a Verilator model, which reports 98.7 % line and branch coverage over the whole regression (`scripts/coverage.sh`).
- **Independent decode.** The waveforms of the standards tests are decoded by sigrok's protocol decoders and checked against independently computed bytes: 30 of 30 pass (`scripts/sigrok_check.py`).
- **Gate level.** The cocotb tests pass on the synthesised and on the routed netlist with the IHP cell models.

7 Implementation results

7.1 Synthesis

Yosys and ABC mapped to the IHP `sg13cmos5l` cells, each block synthesised on its own:

Block	Cells
Frame chain	5,907
of which CRC unit	846
Line codec	1,597
Host FIFO (8 entries)	319
One channel (chain, codec, host FIFO, dispatcher)	8,707
Control unit	961
UART receiver with auto-baud	434
Return path with UART transmitter	324
Whole design (two channels)	**19,145**

Flattened, as `scripts/synth_ihp.sh` does it, the design is 19,721 cells and 333,138 μm^2, 36 % of the 6 × 4 tile area before the flow adds buffers and the clock tree.

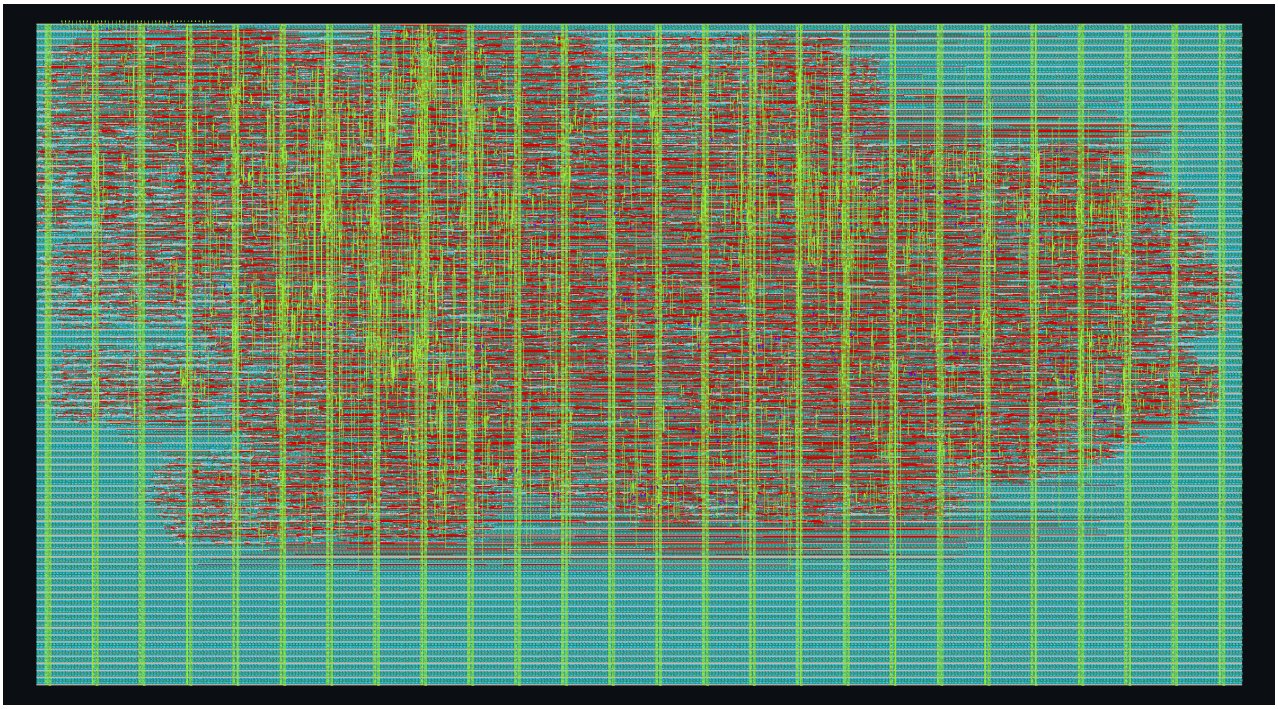
7.2 Place and route on Tiny Tapeout

The design was hardened on the Tiny Tapeout CMOS5L template (the `cmos5l` branch of the IHP template) with the flow its `gds` workflow runs: LibreLane 3.1.0.dev3 and the IHP SG13CMOS5L PDK at the commit the workflow installs, at the 6 × 4 tiles the Jane Street challenge sets. Timing is from the routed design with extracted parasitics.

Tiles	6 × 4, 1289.28 × 710.64 μm
Cells	18,969 after synthesis; 25,612 after place and route, not counting fill (2,763 flip-flops, 6,104 timing repair buffers, 362 clock tree cells)
Utilisation	43 % of the core area
Routing	0 violations after detailed routing, 0 antenna violations (51 diodes)
Sign-off	Magic DRC 0 errors; LVS clean
Wire length	1.15 m

Corner	Setup slack at 50 MHz	Hold slack	Maximum clock	Power
Typical, 1.2 V, 25 °C	9.82 ns	0.29 ns	98 MHz	10.2 mW
Slow, 1.08 V, 125 °C	3.84 ns	0.61 ns	62 MHz	8.1 mW
Fast, 1.32 V, −40 °C	12.35 ns	0.10 ns	131 MHz	12.7 mW

The CMOS5L flow only enforces timing at the typical corner, so `src/config.json` adds three settings to make the slow corner pass as well: a 0.75 ns limit on signal transitions, 3 ns resizer setup margins, and a second setup repair after global routing. The first layout of v0.7.1 still missed the slow corner by 3.8 ns, on paths ending in the CRC register and the frame buffer. Four RTL changes closed it without changing what the pins do, apart from the CRC and reply starting one clock later: the CRC input is registered; MAIN keeps the bits left in a register; the host FIFO head and the frame buffer's fill level are registered, and the buffer writes its memory one clock later with a bypass; and the CRC update picks its length once, after all 8 steps. The verification report gives the slack after each step. Most of the power is the clock reaching the 2,763 flip-flops.



Routed layout, 6 × 4 tiles. Cells cover 43 % of the core; the rest is filler and decoupling cells.

7.3 Pin map

Pin	Direction	Use
`ui[0]`	in	RX: instructions from the host (first byte `0x55`)
`uo[0]`	out	TX: data and status to the host
`uo[1]`	out	BAUD_LOCKED
`uo[2]`, `uo[3]`	out	ARMED for channels A and B
`uo[4]`, `uo[5]`	out	HOST_FULL for channels A and B
`uo[6]`, `uo[7]`	out	SELECT (chip select) for channels A and B
`uio[0]`–`uio[2]`	in/out	pins 0–2, driven by channel A: data, pair, clock
`uio[3]`–`uio[5]`	in/out	pins 3–5, driven by channel B: data, pair, clock
`uio[6]`, `uio[7]`	in	pins 6 and 7
`ui[7:1]`	in	unused

The `PIN` fields count the `uio` pads. Every pad is read back through `pin_in`, so open-drain I^2C and the USB pair work on single pads; a channel drives its pads only when `OUT` enables them.

8 Limits and future work

1. **Return link bandwidth.** Two bytes per host entry. At 4 clocks per bit the link keeps up with USB low speed and the clocked standards. Full-size 10BASE-T frames (64 bytes and more) arrive faster; they are received but overflow the 16-entry buffer and are reported with the overflow flag.
2. **Feeding fixed-timing lines.** `DATA` streams during a frame, so packet length is no longer limited by the 8-entry host FIFO, but each entry costs 140 clocks on the instruction link at 7 clocks per bit. UART transmit keeps up to about 1.2 Mbaud; faster fixed-timing lines must be preloaded (8 entries) or end with an underrun.
3. **Head-of-line blocking.** A host entry waiting for the return link holds up the wire entries

behind it. I²C just stretches SCL; a timed transmit would pause. The appended CRC and the reply are exempt.

4. **Manchester limits.** Receive needs at least 5 clocks per bit (50 MHz for 10BASE-T); transmit needs an even divider (40 or 60 MHz). The routed design's slowest path, 16.2 ns at the slow corner, also fits a 60 MHz clock (16.7 ns), but the flow was only run at 50 MHz.
5. **Not supported yet.** Clock stretching as an I²C target; a *loop* action for `BRANCH` (repeat a block without the host); a CRC over Manchester lines with a reply gap (the gap needs a non-Manchester timed line).
6. **I²C target speed.** Tested up to 1 MHz SCL. Faster, the ACK after a repeated START can be late, because the previous frame's status entry must first reach the return link (head-of-line blocking).
7. **Reset.** `CLEAR` resets a channel through its asynchronous reset (`rst_n && !clear`), driven from a control-unit flip-flop.

9 Files

Folder or file	Content
`info.yaml`	Tiny Tapeout project file: title, tile count, source list, pin names
`src/top/`	`tt_um_vertrex_star_proem.sv` (Tiny Tapeout top level and pin map), `protocol_emulator.sv` (chip core) and `protocol_emulator_inst.sv` (instantiation template)
`src/core/`	`emulator_channel`, `control_unit`, `line_codec`, `entry_dispatch`, `return_path`
`src/core/frame_chain/`	`frame_chain` and its blocks: `first_counter`, `header_counter`, `start_router`, `branch_unit`, `crc_unit`, `main_counter`, `end_counter`, `watchdog`
`src/primitives/`	Reusable building blocks: `uart_rx`, `uart_tx`, `host_fifo`, `frame_buffer`
`src/files.f`	File list in compile order, used by the scripts
`src/config.json`	LibreLane settings for the Tiny Tapeout GDS flow
`test/`	cocotb test bench (`tb.v`, `test.py`) for the RTL and the gate-level netlist
`sim/`	Test bench `bench.h` and the 16 tests `t_*.cpp`
`sim/verilator/`	Adapter that runs the same tests on a Verilator model, for coverage
`formal/`	Formal harnesses and `.sby` files for six modules
`lint/`	`waivers.vlt`, the reviewed Verilator warnings
`scripts/`	`run.sh` (every test, then synthesis), `lint.sh`, `coverage.sh`, `sigrok_check.py`, `formal.sh`, `synth_ihp.sh` (IHP cell count and area), `sta.sh` (timing estimate)
`host/`	Python host library and its README
`gds/`	The routed layout (GDS, LEF), pictures and a results summary
`docs/`	This document, the ISA, the module references, the standards guide, the toolchain guide, the verification report, the diagram and its generator; `docs/README.md` lists them

`bash scripts/run.sh` needs Yosys and g++. Setting `LIBERTY` to an IHP liberty file also reports area. Every source file starts with a header giving the file, module, type (top level, core RTL or primitive), place in the hierarchy and submodules. The RTL is SystemVerilog 2008 in the project's coding style, checked with Yosys, Verilator and Icarus Verilog.